Modelo relacional y restricciones de integridad

Oct 26, 2026 · @Pachi

Modelo relacional

El MER se traduce en 8 tablas. Notación: **negrita** = clave primaria, *itálica* = clave foránea.

- **USUARIO** (**email**, contraseña_hash, nombre, apellido, tipo)
- **PACIENTE** (***email***, documento, telefono)
    - email → USUARIO(email)
- **ESPECIALIDAD** (**id_especialidad**, nombre, descripcion)
- **PROFESIONAL** (***email***, matricula, nombre, apellido, *id_especialidad*)
    - email → USUARIO(email)
    - id_especialidad → ESPECIALIDAD(id_especialidad)
    - nombre, apellido: copia de USUARIO, solo para mostrar en listados (ADR 004)
- **SERVICIO** (**id_servicio**, nombre, *id_especialidad*)
    - id_especialidad → ESPECIALIDAD(id_especialidad)
- **DISPONIBILIDAD** (**id_disponibilidad**, *email_profesional*, fecha, horario_inicio, horario_fin)
    - email_profesional → PROFESIONAL(email)
- **TURNO** (**id_turno**, *email_paciente*, *id_disponibilidad*, *id_servicio*, estado, fecha_creacion, fecha_cancelacion, motivo_cancelacion, cancelado_por, email_profesional, fecha, horario_inicio, horario_fin)
    - email_paciente → PACIENTE(email)
    - id_disponibilidad → DISPONIBILIDAD(id_disponibilidad)
    - id_servicio → SERVICIO(id_servicio)
    - email_profesional, fecha, horario_inicio, horario_fin: copia de la DISPONIBILIDAD al reservar, para el plazo de cancelación, la agenda y el mail (ADR 006)
- **NOTIFICACION** (**id_notificacion**, *id_turno*, tipo, estado_notificacion, destinatario)
    - id_turno → TURNO(id_turno)

Cada tabla vive en el esquema de un solo servicio, dentro de una única instancia de PostgreSQL:

Servicio	Esquema	Tablas
Usuarios	usuarios	USUARIO, PACIENTE
Catálogo	catalogo_medicos	ESPECIALIDAD, PROFESIONAL, SERVICIO, DISPONIBILIDAD
Reservas	reservas	TURNO
Notificaciones	notificaciones	NOTIFICACION

Decisiones del pasaje

Cuatro decisiones explican cómo se llegó de cada elemento del MER a estas tablas.

- **Especialización:** una tabla para el supertipo y una por subtipo, con la misma clave (`email`) como PK y FK. Mantiene los datos comunes en un solo lugar y deja a cada subtipo sus atributos propios.
- **Relaciones 1:N:** cada una se resolvió con una FK del lado N. No hace falta ninguna tabla intermedia porque el MER no tiene relaciones N:M.
- **Participación total del lado N:** se traduce en que la FK es `NOT NULL`. Todo profesional tiene especialidad, y todo turno tiene paciente, disponibilidad y servicio.
- **Copias entre servicios:** PROFESIONAL guarda el nombre y apellido de USUARIO, y TURNO guarda el profesional, la fecha y el horario de su DISPONIBILIDAD. Se copian al crearse para que los listados, la cancelación y la agenda no dependan de otro servicio (ADR 004 y 006).

Restricciones de integridad

Las restricciones se agrupan en seis tipos; las del último grupo no se resuelven solo con PK, FK o CHECK.

1. Integridad de entidad

Todas las claves primarias son únicas y no nulas.

2. Integridad referencial

FK	Referencia	NOT NULL	Al borrar	Cómo se garantiza
PACIENTE.email	USUARIO	sí (es PK)	CASCADE	FK en la base (mismo servicio)
PROFESIONAL.email	USUARIO	sí (es PK)	CASCADE	Aplicación: alta con compensación (ADR 002). El CASCADE queda pendiente: no existe borrar usuarios
PROFESIONAL.id_especialidad	ESPECIALIDAD	sí	RESTRICT	FK en la base
SERVICIO.id_especialidad	ESPECIALIDAD	sí	RESTRICT	FK en la base
DISPONIBILIDAD.email_profesional	PROFESIONAL	sí	RESTRICT	FK en la base
TURNO.email_paciente	PACIENTE	sí	RESTRICT	Token JWT de tipo PACIENTE
TURNO.id_disponi	DISPONIBILIDAD	sí	RESTRICT solo	Reservas consulta a

FK	Referencia	NOT NULL	Al borrar	Cómo se garantiza
bilidad			con turno RESERVADO	Catálogo al reservar. Para borrar una disponibilidad, Catálogo pide a Reservas un bloqueo, que se rechaza si hay un turno RESERVADO (ADR 003). Los turnos cancelados conservan su copia del horario (ADR 006)
TURNO.id_servicio	SERVICIO	sí	RESTRICT	Reservas consulta a Catálogo al reservar
NOTIFICACION.id_turno	TURNO	sí	CASCADE	Llega en el evento del outbox (ADR 005)

Se usa RESTRICT donde borrar dejaría turnos huérfanos o perdería historial. Por ejemplo, una disponibilidad con un turno RESERVADO no se borra; una que solo tiene turnos cancelados sí, porque cada turno conserva una copia de su horario (ADR 006).

3. Integridad de dominio

- USUARIO.tipo ∈ {PACIENTE, PROFESIONAL}
- TURNO.estado ∈ {RESERVADO, CANCELADO, FINALIZADO}
- TURNO.cancelado_por ∈ {PACIENTE, PROFESIONAL}
- NOTIFICACION.tipo ∈ {CONFIRMADO, CANCELADO}
- NOTIFICACION.estado_notificacion ∈ {PENDIENTE, ENVIADO, FALLIDO}
- NOTIFICACION.destinatario ∈ {PACIENTE, PROFESIONAL}
- `fecha` (de DISPONIBILIDAD y su copia en TURNO) es fecha; `horario_inicio` y `horario_fin` son hora; `fecha_creacion` y `fecha_cancelacion` son fecha/hora con zona horaria.

4. Claves candidatas (UNIQUE)

- PACIENTE.documento
- PROFESIONAL.matricula
- ESPECIALIDAD.nombre
- SERVICIO (id_especialidad, nombre)
- DISPONIBILIDAD (email_profesional, fecha, horario_inicio)
- NOTIFICACION (id_turno, tipo, destinatario): como máximo una notificación por turno, tipo y destinatario

5. Restricciones de tupla (CHECK)

- DISPONIBILIDAD: horario_inicio < horario_fin.

- TURNO: si estado = CANCELADO, fecha_cancelacion y cancelado_por no son nulos; si no, fecha_cancelacion, motivo_cancelacion y cancelado_por son nulos.
- TURNO: fecha_cancelacion ≥ fecha_creacion.
- USUARIO, PROFESIONAL y TURNO: el email se guarda en minúsculas (email = lower(email); en TURNO, email_paciente).

6. Restricciones generales

Abarcan varias filas o tablas.

- **Un solo turno activo por horario:** para cada id_disponibilidad hay como máximo un TURNO con estado = RESERVADO. Es la regla clave del control de concurrencia. En PostgreSQL se implementa con un índice único parcial (`UNIQUE (id_disponibilidad) WHERE estado = 'RESERVADO'`): si dos pacientes confirman a la vez, la base rechaza al segundo.
- **Sin superposición de horarios:** dos disponibilidades del mismo profesional en la misma fecha no se superponen. En PostgreSQL se implementa con una restricción `EXCLUDE` sobre el rango horario (extensión btree_gist); una franja puede empezar justo cuando termina otra.
- **Especialización total y disjunta:** todo USUARIO con tipo = PACIENTE está en PACIENTE y no en PROFESIONAL, y viceversa. Como USUARIO y PROFESIONAL viven en servicios distintos, el alta de un profesional se hace con compensación (ADR 002).
- **Coherencia del servicio:** el servicio de un turno pertenece a la especialidad del profesional dueño de la disponibilidad (SERVICIO.id_especialidad = PROFESIONAL.id_especialidad). La valida Reservas consultando a Catálogo al reservar.
- **Plazo de cancelación según quién cancela:** si cancela el paciente (cancelado_por = PACIENTE), solo cuando fecha_cancelacion + 24 h < fecha + horario_inicio del turno. Si cancela el profesional (cancelado_por = PROFESIONAL), en cualquier momento antes de fecha + horario_inicio (ADR 007).
- **Participación total de TURNO en notifica:** todo turno tiene al menos una notificación (la de confirmación). Una FK no lo garantiza, así que lo asegura la aplicación: el evento turno.confirmado se guarda en un outbox en la misma transacción que el turno, y Notificaciones, al consumirlo, crea una NOTIFICACION para el paciente y otra para el profesional (consistencia eventual, ADR 005).
- **Nada en el pasado:** no se crean disponibilidades ni se reservan turnos con fecha + horario_inicio anterior al momento actual, en la zona horaria de Montevideo.
- **Borrado de horarios:** no se borra una disponibilidad con un turno RESERVADO ni una que ya empezó. Reservas lo garantiza con un bloqueo por horario (tabla técnica `disponibilidad_bloqueada`) y un lock de PostgreSQL que comparten reservar y bloquear, así una reserva y un borrado simultáneos no pueden terminar los dos (ADR 003).
- **Copias coherentes:** los atributos copiados toman el valor de su origen al crearse. Las disponibilidades no se editan: modificar un horario es eliminarlo y crear otro (ADR 003).
- **FINALIZADO:** está en el dominio pero hoy nada lo asigna; queda fuera de alcance (ADR 007).

Las que no se resuelven con índices o `EXCLUDE` se implementan con triggers o en la lógica de los servicios.

Claves foráneas entre servicios

Cada servicio tiene su propio esquema y ninguno accede al de otro, así que el motor no puede validar las FK que cruzan servicios. Por ejemplo, TURNO.email_paciente vive en el servicio de Reservas pero apunta a USUARIO, que está en el servicio de Usuarios.

Esas referencias se validan por API, por el token o por eventos; la columna «Cómo se garantiza» de la tabla de integridad referencial dice cuál usa cada una. Esto se conecta con la característica de consistencia que se analiza en el punto 2 del proyecto.